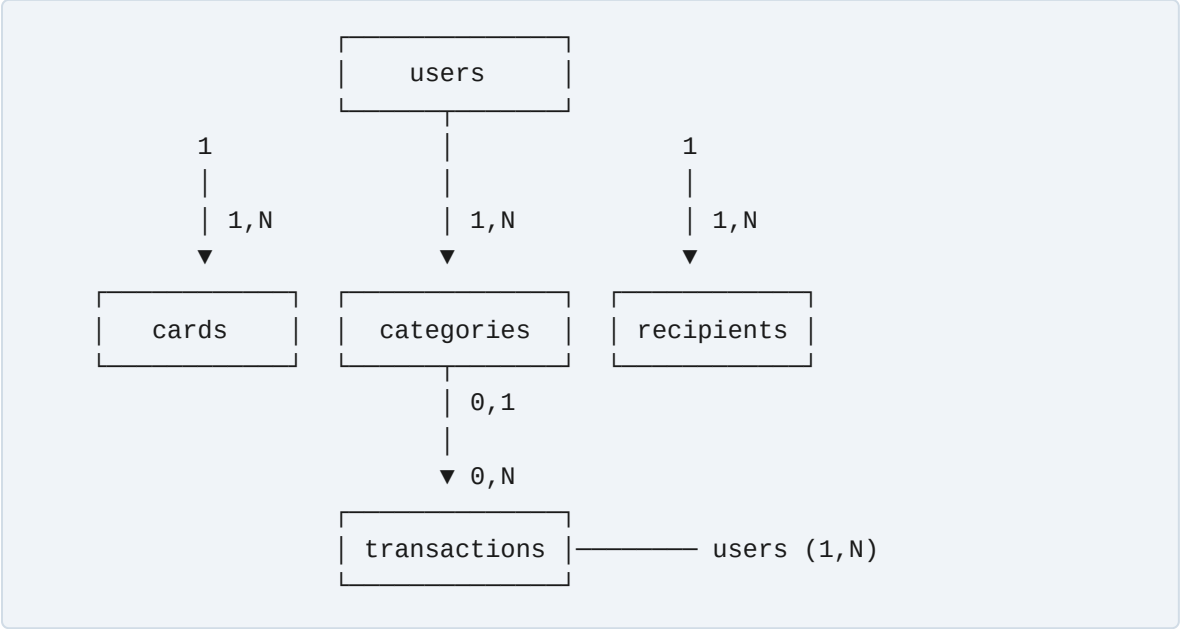
# Relations entre les entités — Budget_App

Livrable de conception (MCD / MLD). Identique pour le Bloc 2 (Node, `Backend_node`) et le Bloc 3 (Symfony, `backend`*`bloc3`*`db`).

---

## 1. Schéma conceptuel (MCD)




En une phrase : **un utilisateur possède ses cartes, ses catégories, ses bénéficiaires et ses opérations**. Une opération** peut** être classée dans **une** de ses catégories (ou aucune).

---

## 2. Cardinalités et clés étrangères

| Relation | Cardinalité | Colonne | ON DELETE | Pourquoi |
|---|---|---|---|---|
| User → Transaction | 1 → N | `transactions.user_id` **NOT NULL** | **CASCADE** | Une opération n'existe que pour un compte. Si on supprime l'utilisateur (RGPD / clôture), on ne garde pas son historique chez les autres. |
| User → Category | 1 → N | `categories.user_id` **NOT NULL** | **CASCADE** | Les libellés (« Courses », « Salaire ») sont **personnels**, pas un référentiel partagé. |
| User → Card | 1 → N | `cards.user_id` **NOT NULL** | **CASCADE** | Même isolation. On ne stocke que `last4digits` + marque + expiration, jamais le PAN. |
| User → Recipient | 1 → N | `recipients.user_id` **NOT NULL** | **CASCADE** | Carnet d'adresses privé (téléphone ou IBAN). Ce n'est **pas** un autre `users.id`. |
| Category → Transaction | 0..1 → N | `transactions.category_id` **NULL** | **SET NULL** | Classer est optionnel. On ne perd pas l'opération si le libellé disparaît. |

Toutes les FK « appartient à » partent de `user_id` lu côté **serveur** (JWT Bloc 2 / `getUser()` Bloc 3), jamais d'un `user_id` envoyé par le navigateur.

---

## 3. Choix que j'ai volontairement refusé

### Pas de `recipientid` *ni* `card` `id` sur `transactions`

Un transfert ou une recharge **écrit une ligne** `income` / `expense` (`title` + `amount`). On ne fige pas le bénéficiaire ou la carte dans la transaction :

- le carnet peut changer (on renomme un contact) sans casser l'historique ;
- le Bloc 2 ne fait pas encore un vrai P2P (pas deux lignes liées) ;
- moins de colonnes = moins de FK à maintenir pour un MVP.

Si on industrialise les virements : alors oui, `recipient_id` nullable + éventuellement une table `transfer_legs`.

### Pas de `users.balance` comme source de vérité

Le solde = `SUM(income) − SUM(expense)` sur **les transactions de cet user**. Stocker `users.balance` **et** les lignes, c'est deux sources qui divergent. La colonne existe encore en Node (dette) ; Symfony ne l'a pas.

### Catégories par user, pas une table globale

« Courses » de A n'est pas « Courses » de B. Ça évite qu'un user voie ou renomme les libellés d'un autre. Prix : un peu de duplication de noms.

### Supprimer une catégorie utilisée

- **Base :** `ON DELETE SET NULL` → l'historique reste, juste non classé.
- **Métier Bloc 2 :** l'API refuse la suppression s'il reste des transactions

(`« N transactions utilisent cette catégorie »`).

- Les deux se complètent : l'API protège l'UX ; la FK protège si on tape

un `DELETE` SQL à la main.

### Recipients ≠ users

`recipients.identifier` est un téléphone ou un IBAN **saisi**, pas une FK vers `users`. En Bloc 2 le destinataire n'est pas un vrai compte. En Bloc 3 le P2P cherche un user par email/téléphone **au moment du transfert**, sans lier le carnet à cet user en dur.

---

## 4. MLD (tables)

```
users (id PK, email UNIQUE, password, role, fullName, …)
categories (id PK, user_id FK→users CASCADE, name)
transactions (id PK, user_id FK→users CASCADE, category_id FK→categories SET
              title, amount, type ENUM income|expense, created_at)
cards (id PK, user_id FK→users CASCADE, card_holder_name, last_4_digits,
       expiry_date, card_brand, created_at)
recipients (id PK, user_id FK→users CASCADE, name, identifier, type ENUM phc
            created_at)
```

Index utiles (déjà en place) : `user_id` sur chaque table fille, `transactions(created_at)`, `transactions(type)`, `transactions(user*id, type, created*at)` pour les stats.

---

## 5. Où c'est créé dans le code

| Bloc | Fichier |
|---|---|
| 2 — CREATE + ALTER (base déjà existante) | `Backend/utils/db.js` (`ensureForeignKeys`) |
| 2 — script manuel phpMyAdmin | `Backend/migrations/add*entity*relations.sql` |
| 2 — garde métier catégorie | `Backend/handlers/categories.js` + `transactions.js` (`assertCategoryOwned`) |
| 3 — Doctrine + SQL | `Backend_bloc3/src/Entity/*` et `migrations/Version20260306142322.php` |

Au démarrage Node, `CREATE TABLE IF NOT EXISTS` ne rajoute **pas** les FK sur une vieille base : `ensureForeignKeys()` fait les `ALTER TABLE` et nettoie les `category_id` orphelins (`SET NULL`).

---

## 6. Phrase d’oral (30 s)

« Cinq tables. Un user a N cartes, N catégories, N bénéficiaires, N transactions. La catégorie sur une transaction est optionnelle : FK `ON DELETE SET NULL`. Tout le reste est `ON DELETE CASCADE` parce que ces lignes n’ont pas de sens sans le compte. Le solde n’est pas une colonne, c’est un agrégat. On n’a pas mis de FK vers la carte ou le contact sur la transaction : le MVP écrit juste une opération. »